AUTHORIZATION SYSTEMS / RESEARCH PROTOTYPE

Auths-Proof: Principal-Agnostic Proof-Carrying Authorization Across Identity and Transport Boundaries

bordumb · bordumbb@gmail.com

24 July 2026

ABSTRACT

Authentication answers who controls a credential; authorization answers whether that principal may perform a particular action. Distributed systems routinely collapse these questions: a mutually authenticated channel, valid token, recognized decentralized identifier, or verified signature is treated as authority. The result is protocol-specific policy, confused trust boundaries, and authorization state that cannot travel with an action. We present **Auths-Proof**, a prototype proof-carrying authorization system built around one architectural rule: **Auths owns authority; adapters prove principal control.** A bounded, deterministic verifier evaluates a signed, attenuating grant chain and an action envelope against a verifier-local trust anchor. Principal methods, signature algorithms, transports, and application profiles are independent ports. Adapters may establish control using a raw key, `did:key`, `did:keri`, `did:web`, or a future method, but they return explicit assurance claims rather than silently equating unlike evidence. Iroh, HTTPS, TCP, Unix sockets, files, and in-memory channels may carry the same proof; channel authentication never creates Auths authority. The V1 artifact demonstrates Ed25519 and P-256, four principal adapters, native and WebAssembly verification, in-memory and Iroh exchange, and an MCP `tools/call` profile. A mixed-method fixture delegates from a rotated KERI root to a raw P-256 agent and produces the same authorization result across transports. Preliminary measurements on one development machine show sub-millisecond native verification and 1.320 ms mean browser verification for a 1,988-byte proof; these are engineering observations, not production benchmarks. We position the system as a deliberately small authorization kernel, identify what it does not solve, and define falsifiable invariants for future security and interoperability evaluation.

1. Introduction

The modern identity stack is rich in ways to establish *who* is present. OpenID Connect authenticates an end user through an authorization server (OpenID Foundation 2023). SPIFFE issues short-lived workload identities for mutual authentication across heterogeneous infrastructure (Cloud Native Computing Foundation 2026). DIDs provide a common identifier and document model over method-specific resolution (Sporny et al. 2022). Iroh lets software dial an endpoint by a public key rather than an IP address (n0 computer 2026). HTTP Message Signatures bind selected HTTP semantics to a key (Backman, Richer, and Sporny 2024).

None of those facts alone answers the application question:

> **Was this exact action authorized by authority I trust, for this resource, audience, challenge, and time?**

That question becomes urgent when actions cross process, service, organization, and time boundaries. An AI agent calls a tool. A build worker publishes an artifact. A controller changes infrastructure. A device acts intermittently at the edge. In each case, the receiver may need to verify authorization without calling the issuer, trusting the delivery path, or sharing the sender's identity technology.

The underlying ideas have deep roots. Proof-carrying code asks an untrusted producer to supply a safety proof that a consumer can check (Necula 1997). Proof-carrying authentication applies the same asymmetry to distributed authentication and authorization logic (Appel and Felten 1999; Bauer, Schneider, and Felten 2002). SPKI binds authorization directly to keys and reduces certificate tuples (Ellison et al. 1999). Decentralized trust management separates policy, credentials, and the decision of whether credentials satisfy policy (Blaze, Feigenbaum, and Lacy 1996). Macaroons

provide efficient attenuating bearer credentials with contextual caveats (Birgisson et al. 2014).

Auths-Proof does not claim to originate proof-carrying authorization, delegation, capabilities, or decentralized identity. Its systems contribution is a stricter composition boundary:

Bring any cryptographic principal. Auths proves whether its action was authorized.

The architectural rule is equally compact:

Auths owns authority. Adapters prove principal control.

This produces four independently replaceable axes, shown in Figure 1. The principal method explains how a claimed principal controls verification material. The signature algorithm verifies a statement. The transport moves opaque proof-bearing actions. The application profile maps domain operations to canonical bytes and exact permissions. Only the authority kernel interprets grants and returns an Auths verdict.

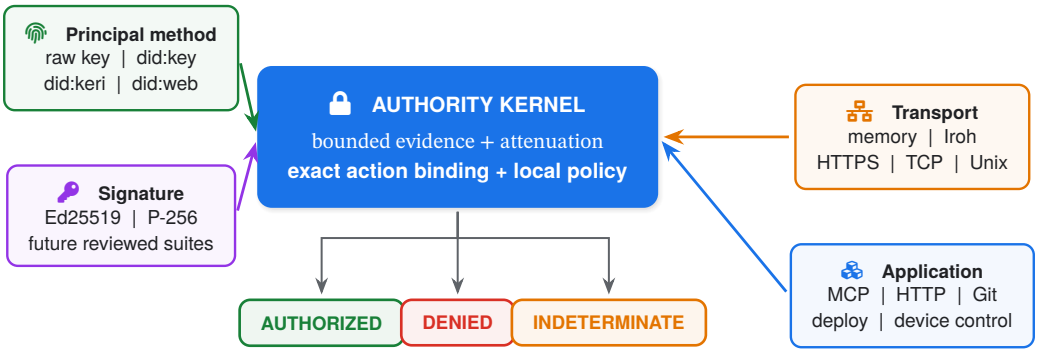

Figure 1: One authority primitive, four substitution axes. Identity technology, cryptographic suite, network, and application semantics are ports. They contribute evidence or context; none may redefine authority. The labels name implemented V1 adapters and plausible future adapters, not a claim that every listed option ships today.

1.1 Contributions

This paper makes five concrete contributions:

1. **A principal-agnostic authority model.** Principal-control adapters return a uniform verified-principal result, while the authority kernel owns grant semantics. A delegation chain may cross principal methods and signature algorithms without translating authority into the identity layer.
2. **Assurance as typed output.** Successful verification is not a Boolean that erases how control was established. Adapters return explicit assurance claims; local policy chooses which claims are sufficient.
3. **Transport non-conflation.** An authenticated Iroh or TLS peer is an observation about the channel, never an Auths authorization fact. Final execution requires the conjunction of Auths, channel-binding, and application policy.
4. **A deterministic, portable verification core.** The kernel has no network, filesystem, process, environment, clock, randomness, private key, database, Git, or async capability. It accepts explicit bytes and context, uses bounded decoding, and targets native and `wasm32-unknown-unknown`.
5. **An executable V1 artifact.** Separate workspaces implement the authority protocol, proof exchange, and one narrow MCP profile, with conformance fixtures spanning rotated KERI, raw P-256, in-memory exchange, Iroh exchange, and browser verification.

1.2 Scope and status

Auths-Proof is a research prototype and protocol-design artifact, not a production security claim. Its current V1 registry is intentionally small: Ed25519 and P-256 signatures; raw-key, `did:key`, `did:keri`, and bundled `did:web` evidence; in-memory and Iroh exchange; and one MCP `tools/call` profile. Architecture-level extensibility means a new adapter can be added without changing authority semantics. It does **not** mean arbitrary algorithms, DID methods, URLs, or transports are accepted automatically. Unknown identifiers fail closed or yield `Indeterminate` according to the specified condition.

2. Problem and design goals

2.1 Authentication is an input, not the decision

Distributed access-control literature has long distinguished a principal making a statement from the policy decision to trust that statement (Abadi et al. 1993). Yet application stacks often reunite them:

- mTLS authenticates a workload, then a service maps the certificate subject directly to permissions.
- A DID resolver returns verification material, then method-specific code performs authorization.
- A transport authenticates a peer key, then the application assumes the peer may call the endpoint.
- A signed message proves control of a key, then the receiver treats the signature as consent for every covered operation.

Each shortcut can work inside one deployment, but it makes the security model implicit and difficult to move. It also turns identity migration into policy migration. If a root rotates from Ed25519 to P-256, a workload moves from X.509 to a self-certifying identifier, or an operation moves from HTTPS to Iroh, the authority story should not be rewritten.

2.2 Design goals

The design follows seven goals.

G1 - Authority stability. Grant meaning and action authorization remain stable when a principal method, algorithm, transport, or application adapter is replaced.

G2 - Exact binding. A proof authorizes one canonical action body, exact permission, resource, audience, challenge, time, actor, and terminal grant.

G3 - Monotonic delegation. A child grant cannot enlarge permission scope, time, or delegation depth. Chain linkage is explicit and signed.

G4 - Verifier sovereignty. Trust anchors and assurance requirements are local verifier inputs. A proof cannot smuggle in its own root of trust or lower the verifier's policy.

G5 - Honest uncertainty. Unsupported or unavailable evidence is not authorization. It is distinct from a cryptographically or semantically invalid proof.

G6 - Offline and portable verification. Once evidence is assembled, the kernel can verify without ambient I/O and can compile to browser WebAssembly (Haas et al. 2017).

G7 - Small semantic surface. Auths-Proof does not become an identity wallet, network stack, policy language, secrets manager, global ledger, generic RPC framework, or application gateway.

2.3 Non-goals

The system does not prove that an authorized principal is benevolent, protect an unsafe trust anchor, assign meaning to misleading permission strings, or make execution transactional with verification. It does not provide confidentiality, principal discovery, private-key custody, global revocation, exactly-once delivery, or a universal replay database. These omissions are security boundaries, not backlog euphemisms.

3. System model

3.1 Principals, grants, actions, and context

Let p be an opaque `PrincipalId`. Its syntax does not grant authority. A principal adapter m establishes control for a verification method, purpose, algorithm, message, signature, and bounded evidence:

$$\mathrm{Control}_m(p, v, u, a, x, \sigma, E) \to \begin{cases} \mathrm{Verified}(p, A) \\ \mathrm{Reject}(r) \\ \mathrm{Unsupported}(r) \end{cases}$$

where A is a set of assurance claims. The adapter may parse KERI events, decode a Multikey, consult a verifier-supplied `did:web` trust record, or verify a raw self-certifying key. It cannot create an Auths grant or verdict.

A grant g_i contains an issuer, subject, exact permission set, validity window, remaining delegation depth, optional audience constraints, and the identifier of its parent. Its signature covers the canonical grant statement. The root

grant is accepted only relative to a verifier-local trust anchor.
An action envelope binds:

$$H(\text{body}),\ (\text{capability}, \text{resource}),\ \text{audience},\ \text{challenge},\ \text{time},\ \text{actor},\ \text{id}(g_n)$$

The verifier context supplies the body bytes, expected audience, challenge, evaluation time, trust anchor, adapter registry, resource limits, and required assurance. No ambient clock or network lookup is performed.

3.2 Proof anatomy

Figure 2 separates *authority statements* from *principal-control evidence*. Evidence is referenced by digest and bounded by type and size. The trust anchor is notably absent from the portable bundle.

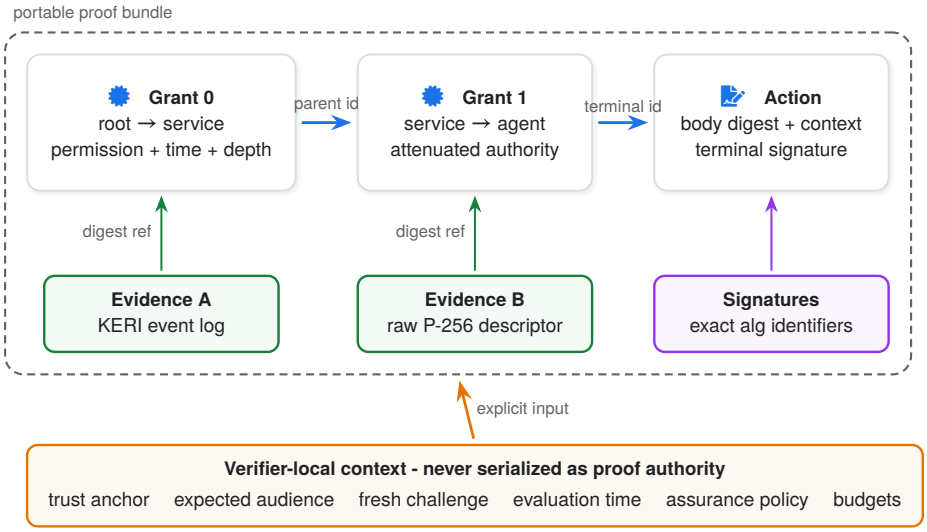

Figure 2: Proof anatomy. Signed grants carry authority; evidence lets an adapter establish control of each issuer or actor. The verifier supplies the trust anchor and expected context. A mixed chain can use KERI at one hop and a raw P-256 key at another without changing grant semantics.

3.3 Exact permissions and attenuation

V1 deliberately avoids a general policy language. A permission is the exact pair:

```
(capability, resource)
```

There are no wildcards, glob rules, negative grants, inherited roles, or application-defined matching functions in the kernel. If P_i is the child permission set, $W_i = [nbf_i, exp_i]$ its validity window, and d_i its remaining delegation depth, every edge must satisfy:

$$P_i \subseteq P_{i-1} \quad \wedge \quad W_i \subseteq W_{i-1} \quad \wedge \quad d_i < d_{i-1}$$

as well as:

$$\text{issuer}(g_i) = \text{subject}(g_{i-1}) \quad \wedge \quad \text{parent}(g_i) = \text{id}(g_{i-1})$$

These checks are intentionally uncreative. An application profile may define a resource such as `mcp://reports/read_report`; it may not redefine subset, validity containment, parent linkage, or the meaning of an Auths verdict.

Figure 3: Authority forms a narrowing funnel. Every delegation edge must be a subset in permission, contained in time, and strictly lower in depth. There is no adapter hook that can broaden this relation.

3.4 Three-valued verdicts

The verifier returns:

- `Authorized`: every cryptographic, structural, authority, action-binding, and required-assurance check passed.
- `Denied`: available evidence establishes that the proof is invalid or the requested action is outside delegated authority.
- `Indeterminate`: the verifier cannot establish a required fact, for example because an exact adapter is unsupported or required freshness evidence is absent.

Collapsing the last two states is operationally tempting but semantically harmful. `Denied` can be a stable policy outcome; `Indeterminate` often means the verifier, evidence package, or deployment is insufficient. More importantly, neither state may be upgraded by transport or application code.

4. Ports, adapters, and security boundaries

4.1 Agnostic does not mean permissive

The word *agnostic* is easy to misuse in cryptographic software. Auths-Proof is agnostic at a stable port and strict at every concrete registry.

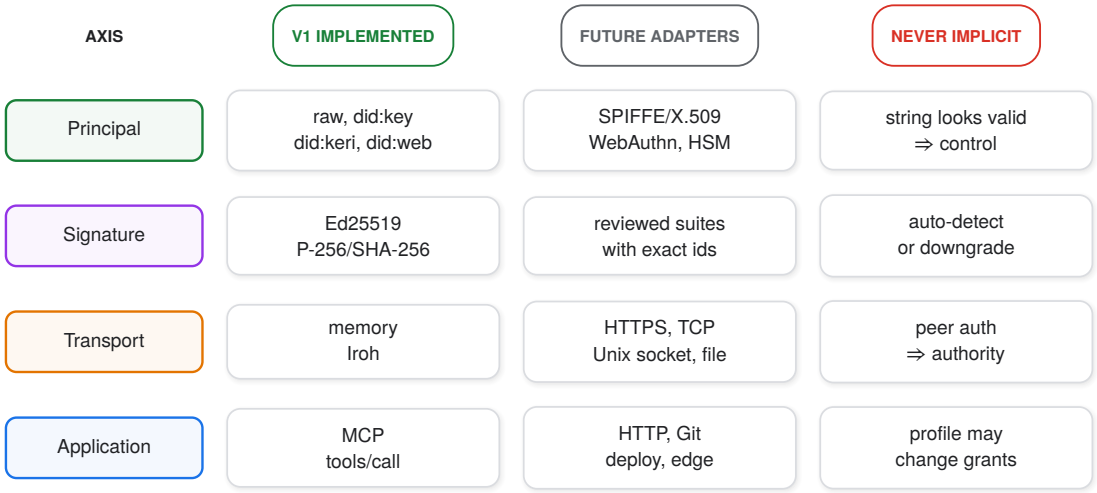

Figure 4: Extensibility is explicit substitution, not algorithmic ambiguity. V1 accepts only registered identifiers and bounded evidence. Future support requires a reviewed adapter and conformance fixtures; fallback and auto-detection are forbidden.

This distinction matters for the user examples. SHA-256 can safely appear as a digest function or self-certifying identifier component, but a bare SHA-256 digest does not itself prove control unless a registered method defines the statement and evidence. SHA-1 must not become security-bearing identity material in a new protocol. Similarly, `did:web` and `did:keri` share DID syntax but establish control under materially different evidence and freshness models.

4.2 Principal-control port

The conceptual Rust boundary is:

```rust
pub trait PrincipalControlVerifier {
    fn verify_control(
        &self,
        request: ControlRequest<'_>,
        evidence: &EvidenceSet<'_>,
        limits: VerificationLimits,
    ) -> Result<VerifiedPrincipal, PrincipalControlError>;
}

pub struct VerifiedPrincipal {
    principal: PrincipalId,
    verification_method: VerificationMethodId,
    assurance: AssuranceClaims,
}
```

The authority engine performs exact adapter lookup from signed metadata. It does not ask each adapter to "try" the bytes. This prevents parser differentials and algorithm confusion from turning fallback order into security policy.

The port also does not expose signing. Verification can be pure and portable; private-key custody belongs to platform-specific authoring adapters, HSMs, wallets, or agents. This is especially important for WebAssembly: browser verification should not drag in OS keychains, network resolvers, or random number generators.

4.3 Assurance-carrying adapters

A verified signature proves only that some verification material accepted a signature. The security significance depends on how that material was bound to the principal and what evidence is current.

V1 models claims including:

```
SelfCertifyingIdentifier    OfflineVerifiable
ControllerStateCurrentAt    HistoricalAt
StatementExistenceProvenAt  RotationAware
RevocationCheckedAt         WitnessThresholdMet
PkiChainValidated           HardwareAttested
```

An adapter may return only claims it actually established. A raw key and `did:key` can be self-certifying and offline verifiable, but have no native rotation or revocation story. A supplied KERI key event log can establish valid rotation history, but without evidence that no later event exists it cannot claim globally current state. A bundled `did:web` trust record can be verified offline, but the verifier must distinguish historical key state from proof that the action existed at that historical time.

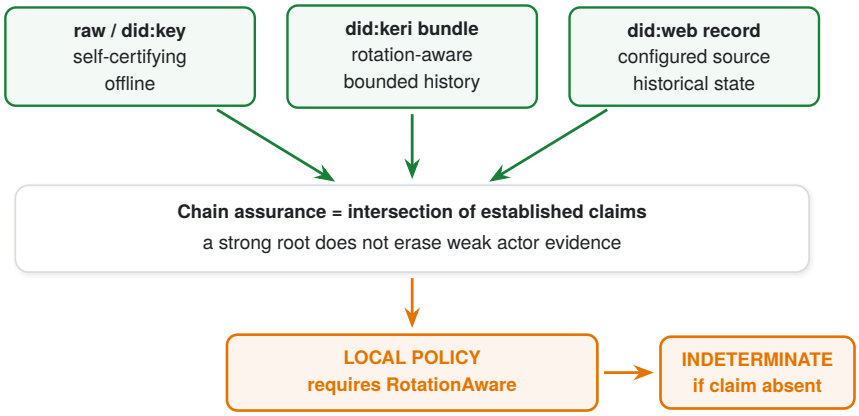

Figure 5: Adapter success does not erase evidence quality. Assurance flows as typed data into local policy. Requirements apply across the chain, so a high-assurance root cannot launder a low-assurance actor.

This mechanism is narrower than a general trust calculus, but it addresses a common integration failure: treating every valid signature or DID resolution as equivalent confidence.

4.4 Algorithm port

Signature algorithms have exact identifiers, key encodings, signature encodings, and verification rules. V1 supports:

Identifier	Public-key encoding	Signature rule
`ed25519`	32-byte compressed Edwards point	64-byte Ed25519 signature
`p256-sha256`	33-byte compressed SEC1 point	fixed V1 ECDSA encoding and low-S normalization

The architectural point is not that these algorithms are interchangeable in cryptographic strength or operational behavior. It is that authority statements do not hard-code either one. A future suite must receive a unique identifier, bounded parser, negative vectors, and explicit compatibility rules. Unknown suites never fall back to a "close enough" verifier.

4.5 Transport port

Transport is a separate protocol layer because networking and authority answer different questions:

- Transport: *How did these bytes arrive, and what did the channel observe?*
- Auths-Proof: *Does this proof authorize this exact action under local trust?*

The proof-exchange protocol therefore carries an opaque action body, opaque Auths proof, challenge, peer observation, and response. It must not parse grants, manufacture verdicts, or promote a peer key into an Auths principal.

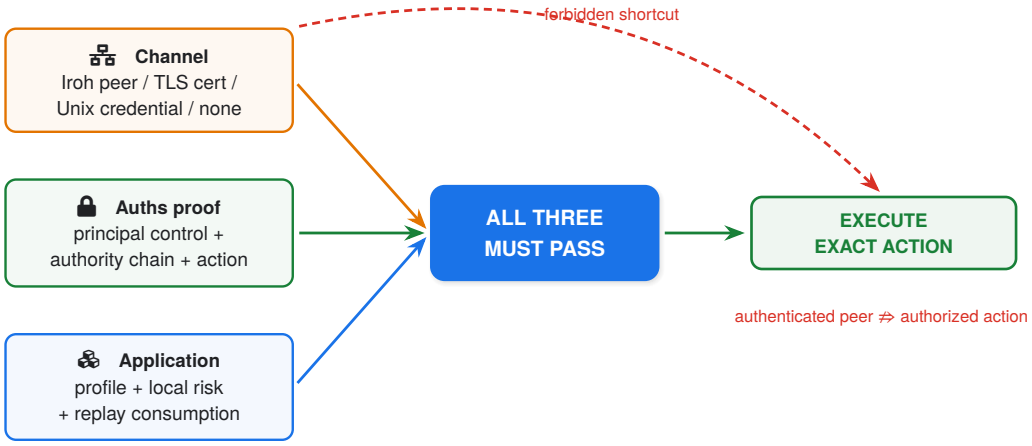

Figure 6: Connectivity never creates authority. An application may require channel binding, but execution is a conjunction. An authenticated Iroh or TLS peer cannot upgrade 'Denied' or 'Indeterminate'.

Iroh is a particularly natural adapter because its endpoint identity is a key and it handles path discovery, hole punching, relay fallback, and QUIC connectivity (n0 computer 2026). But making Iroh the protocol would repeat the coupling the system is designed to remove. The same semantic exchange can ride HTTPS, TCP, a Unix socket, a file, or an in-memory channel. Iroh endpoint keys and Auths principals remain separate by default.

4.6 Application profile port

An application profile owns the mapping from domain operation to canonical body, capability, resource, audience, and challenge handling. The MCP V1 profile maps:

```
operation   = tools/call
capability  = mcp.tools.call
resource    = mcp://<service>/<tool>
```

```
audience    = mcp://<service>
body        = RFC 8785 canonical JSON
```

JCS makes the JSON hash stable across producers (Rundgren, Jordan, and Erdtman 2020). The profile does not authorize generic MCP, a server, or all tools. A grant for one tool cannot authorize another because the exact resource and canonical body are signed.

5. Verification protocol

5.1 Deterministic encoding and domain separation

The portable proof uses a constrained deterministic CBOR profile (Bormann and Hoffman 2020). Every signed object has a domain-separated preimage that includes protocol family, version, object type, and canonical content. The decoder rejects:

- duplicate or unknown critical fields;
- non-canonical integer or collection encodings;
- oversized byte strings, arrays, maps, evidence sets, or grant chains;
- unregistered adapter, algorithm, purpose, or media-type identifiers;
- references to missing evidence or digest mismatches;
- trailing bytes and ambiguous alternative representations.

Canonicalization is not cosmetic. If two byte sequences can express the same semantic grant, identity and signature verification can disagree about what was authorized. V1 computes object identifiers from canonical bytes and signs a domain-separated form.

5.2 Verification sequence

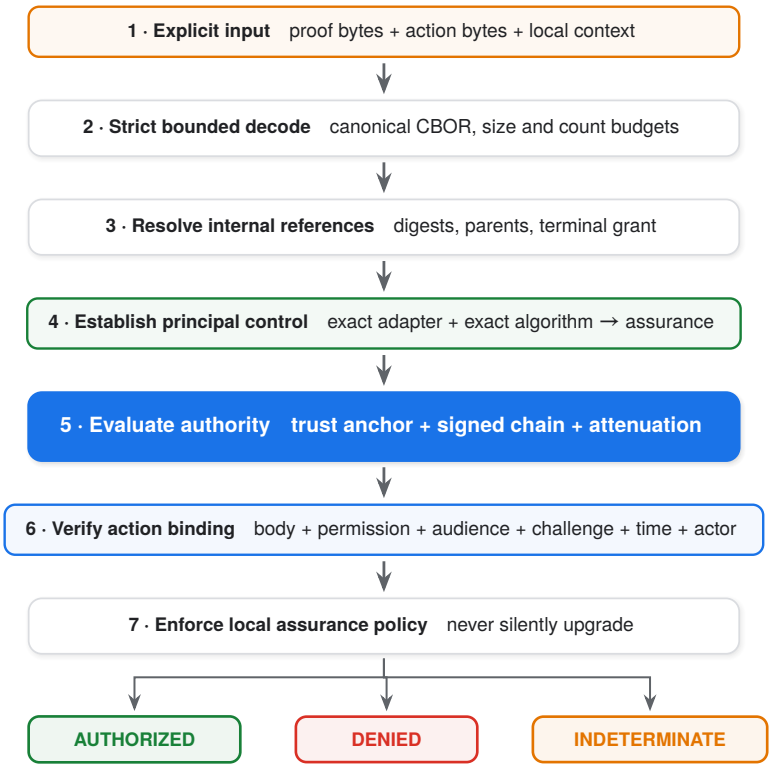

Figure 7: Fail-closed verification pipeline. The pure verifier receives all ambient facts explicitly. Principal adapters establish control and assurance; only the kernel interprets authority.

The ordering is security relevant. Structural rejection occurs before expensive cryptography. Evidence references are resolved before adapters execute. Authority is checked before action execution but after principal control. Application code receives a final typed result and may further restrict it; it may never broaden it.

5.3 Authorization predicate

For proof π, body b, context c, trust anchor r, and local assurance requirement Q:

$$\begin{aligned}\text{Authorized}(\pi, b, c, r, Q) \iff\ & \text{Canonical}(\pi) \wedge \text{Bounded}(\pi) \wedge \\ & \text{AnchoredChain}(\pi, r) \wedge \text{ControlValid}(\pi) \wedge \\ & \text{Attenuating}(\pi) \wedge \text{ActionBound}(\pi, b, c) \wedge \\ & Q \subseteq \text{CommonAssurance}(\pi).\end{aligned}$$

`CommonAssurance` is conservative across required chain participants. The model does not let a KERI root's rotation evidence imply that a delegated raw key is itself rotation-aware.

5.4 Security invariants

I1 · Authority isolation

Only the authority kernel can construct an Auths verdict. Principal, cryptographic, transport, and application adapters return typed observations or errors.

I2 · Principal-method substitution

If two adapters establish the same principal-control statement and sufficient assurance for the same signed bytes, replacing one with the other does not change grant attenuation or action-binding semantics.

I3 · Transport invariance

For identical proof bytes, action bytes, and verifier context, the Auths verdict is independent of whether delivery used memory, Iroh, HTTPS, TCP, a Unix socket, or a file. A transport may independently reject; it cannot authorize.

I4 · No ambient authority

Network reachability, an authenticated peer, current process identity, local filesystem state, environment variables, and wall-clock reads are not implicit inputs to the kernel.

I5 · Monotonic authority

No accepted child grant expands permissions, time, or delegation depth, and no action can exceed its terminal grant.

These are design and test invariants, not machine-checked theorems. A future formalization should encode the grant relation and prove transport non-interference over an abstract adapter result. The current artifact uses workspace dependency checks, property tests, negative fixtures, and cross-target conformance to make violations observable.

6. Implementation

6.1 Three workspaces, three reasons to change

The prototype is split across separately versioned Rust workspaces:

1. `auths-proof` owns the model, codec, authority kernel, principal adapters, authoring helpers, CLI, WebAssembly bindings, fixtures, and architecture checks.

2. `auths-proof-exchange` owns transport-neutral challenge/submission/response semantics plus in-memory and Iroh adapters.
3. `auths-proof-mcp` owns the narrow MCP `tools/call` profile and composes the first two workspaces.

"Separately versioned" means separate repositories or release units with their own manifests and compatibility promises, not branches of one repository. This keeps the authority protocol usable without networking or MCP, and lets an Iroh upgrade avoid forcing a new proof-format release.

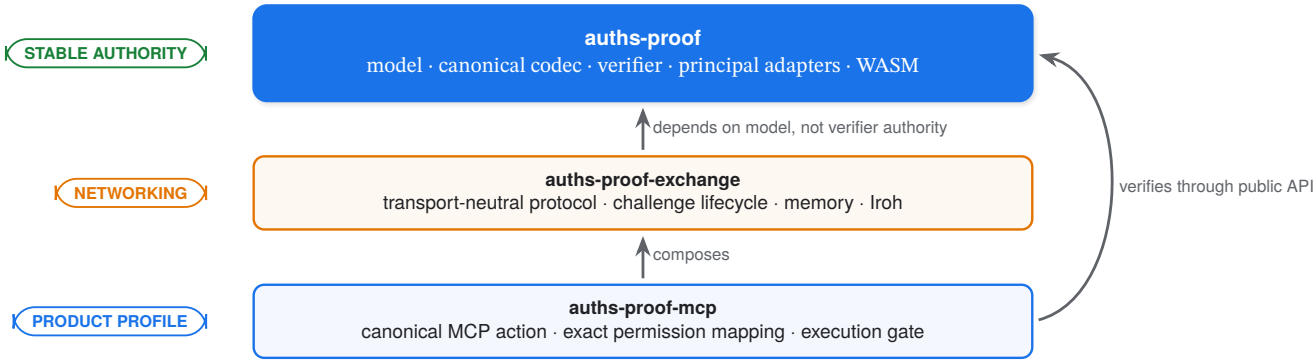

Figure 8: Dependency direction follows semantic ownership. The authority kernel does not depend on transport or MCP. Exchange moves opaque proofs. MCP narrows domain semantics and executes only after all gates pass.

Within the core workspace, crate boundaries follow capability:

Layer	Representative crates	Allowed responsibility
Model	`auths-proof-model`	Validated newtypes, grants, actions, verdicts
Wire	`auths-proof-codec`	Deterministic bounded CBOR
Port	`auths-proof-principal`	Principal-control verification contract
Kernel	`auths-proof-verifier`	Authority, attenuation, action binding
Adapters	`auths-proof-raw-key`, `-did-key`, `-did-keri`, `-did-web`	Exact method evidence
Authoring	authoring and CLI crates	Explicit signing workflows, no verifier secrets
Surfaces	CLI, WASM	Presentation and serialization only
Assurance	testkit, `xtask`	Fixtures, dependency rules, conformance

`xtask` inspects Cargo metadata to prevent the kernel from acquiring forbidden dependencies or platform capabilities. CI builds native and `wasm32-unknown-unknown`, runs formatting and lints, executes conformance and property tests, scans dependency policy, and validates that generated fixtures remain canonical.

6.2 KERI as an adapter, not the kernel

KERI is useful because key event logs can express inception, rotation, pre-rotation commitments, thresholds, and witnessed receipts (Smith 2019). V1 implements a deliberately bounded subset sufficient for offline control verification: inception and rotation validation, sequence and digest continuity, signing thresholds, Ed25519 and P-256 keys, and selected CESR encodings.

The adapter does not export KERI events into grant semantics. It produces:

```
VerifiedPrincipal {
  principal: did:keri:...,
```

```
  verification_method: ...,
  assurance: { SelfCertifyingIdentifier,
               OfflineVerifiable,
               RotationAware }
}
```

This design also contains KERI's limitations. A supplied event log can prove that its included rotation chain is valid. In an offline setting it generally cannot prove that no later rotation or revocation exists. Witness or transparency evidence would justify stronger claims, much as key-transparency systems make directory equivocation detectable (Melara et al. 2015) and Certificate Transparency makes certificate issuance publicly auditable (Laurie, Langley, and Kasper 2013). Until such evidence is implemented, the adapter does not claim global currentness.

6.3 WebAssembly boundary

The browser verifier shares the pure model, codec, adapters, and authority kernel with native Rust. Network-backed resolution is not hidden behind conditional compilation. For example, `did:web` is split into:

- a pure adapter that verifies a bundled, verifier-approved trust record; and
- a native resolver outside the kernel that may fetch and prepare that record.

This avoids a common cross-platform failure: one nominal verifier with materially different trust behavior under `cfg(target_arch = "wasm32")`. The same bundle and context should produce the same verdict on native and WASM.

6.4 MCP over memory and Iroh

The Milestone 4 fixture exercises the composition:

```
rotated did:keri root
    -> delegates reports/read_report
raw P-256 agent
    -> signs canonical MCP tools/call
same 1,988-byte Auths proof
    -> in-memory exchange
    -> direct local Iroh exchange
    -> browser WASM verification
```

The exchange protocol begins with a server challenge. The submission binds that challenge and canonical body. The application atomically claims the challenge, then checks profile, channel policy, Auths verdict, exact permission, and local policy before executing. Iroh V1 uses a dedicated ALPN and a full handshake; it does not use 0-RTT for authorization-bearing submissions.

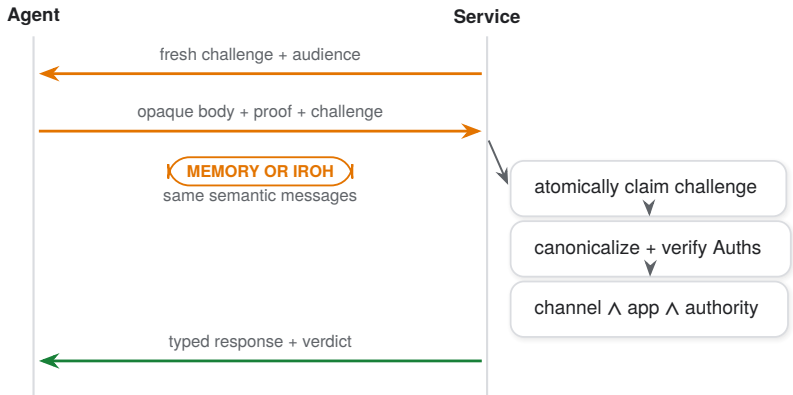

Figure 9: Challenge-bound proof exchange. Replay consumption is an application responsibility adjacent to execution. The transport carries the same messages; it does not inspect authority.

7. Preliminary evaluation

The evaluation asks four prototype questions:

1. Can one grant chain cross principal methods and algorithms?
2. Does the same proof produce the same verdict across transport adapters?
3. Does the verifier run unchanged in a browser?
4. Is verification latency plausibly small relative to a local network exchange?

The checked-in fixture answers these questions for one narrow scenario. It uses a rotated KERI root and raw P-256 agent, succeeds in-memory and over direct local Iroh, and verifies in Chrome WebAssembly.

7.1 Measurements

Measurements were recorded on an Apple M1 Max development machine with Rust 1.94. The proof is 1,988 bytes.

Path	End-to-end	Auths verification	Scope
In-memory	543 us	454 us	challenge, verify, static execution
Iroh direct, local	30.489 ms	505 us	connection plus proof exchange
Chrome 150 WASM	n/a	1.320 ms mean	100 verification iterations

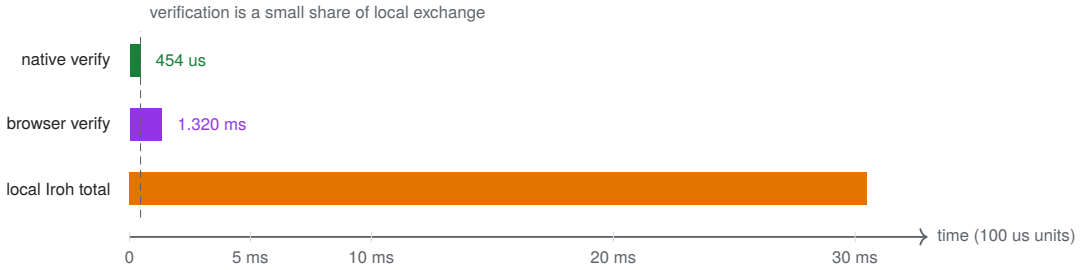

Figure 10: Illustrative latency, not a benchmark claim. The horizontal scale is linear. Native and browser verification are small relative to connection setup in this local Iroh run. No WAN, relay, concurrency, tail latency, or adversarial-input result is implied.

These numbers are useful as a feasibility check only. The runs were not a controlled benchmark campaign; there are no confidence intervals, multi-host trials, cold/warm separation, relay paths, or load tests. A publishable performance evaluation must add those controls and include malformed-proof costs, worst-case chain and evidence limits, memory usage, and browser/device diversity.

7.2 Conformance evidence

The stronger current evidence is semantic:

- mixed-method and mixed-algorithm grant fixtures;
- native and WASM outcome parity;
- identical application results over in-memory and Iroh transports;
- negative fixtures for mutation, reordering, wrong audience, wrong challenge, expired time, widened permission, broken parent link, unsupported adapter, and missing assurance;
- architecture checks that reject forbidden dependency edges;
- bounded parsers and property tests around canonical round trips.

The next evaluation milestone should publish a versioned conformance corpus that independent implementations can consume. Interoperability between two codebases is more meaningful than more tests in one repository.

8. Security analysis

8.1 Threat model

The adversary may control the network, replay or reorder bundles, mutate evidence, choose malformed encodings, present unsupported identifiers, mix principal methods and algorithms, and supply a valid proof for a different action. The adversary may also authenticate successfully at the transport layer.

The trusted computing base includes:

- the verifier binary and selected adapters;
- cryptographic implementations;
- the verifier-local trust anchor, context, budgets, and assurance policy;
- canonical body construction in the application profile;
- the execution gate and challenge/replay store;
- any resolver or evidence assembler whose output local policy elects to trust.

8.2 Attacks addressed

Proof substitution. The body digest, exact permission/resource, audience, challenge, actor, time, and terminal grant prevent moving an action signature to a different call or context.

Authority escalation. Subset, time-containment, strict-depth, issuer/subject, and parent-ID checks reject broadened or spliced chains.

Algorithm and adapter confusion. Exact identifiers and no fallback prevent malformed evidence from being interpreted by a more permissive adapter.

Trust-anchor injection. The root of trust is local context, not a proof field.

Transport identity confusion. Peer observations are typed separately and cannot produce `Authorized`.

Assurance laundering. Claims are explicit and conservative across the chain. Missing required evidence yields `Indeterminate`.

Resource exhaustion. The wire profile places limits on total bytes, nesting, collections, grants, evidence, and adapter work before or during verification.

8.3 Attacks not addressed

> **Authorization is not safe execution.** A compromised authorized key can authorize malicious actions. A correct proof can still name a dangerous capability. A service can verify one body and execute another. A consumed challenge can race with side effects. Auths-Proof narrows and authenticates authority; the host must preserve the verify-to-execute binding.

The system also does not stop:

- compromise of the trust anchor, verifier, adapter, resolver, or application;
- side channels or implementation bugs in cryptographic libraries;
- globally stale KERI or `did:web` state without a freshness source;
- denial of service below configured limits;
- metadata leakage from proof contents or transport;
- key exfiltration in authoring systems;
- semantic collision caused by a poorly designed application profile;
- cross-service replay when audiences or challenges are misconfigured;
- rollback to an older protocol version outside negotiated policy.

8.4 The offline freshness limit

An offline bundle can prove that included statements and event chains are internally valid. It cannot, by itself, prove that no newer revocation or rotation exists. This is not unique to KERI: certificate status, transparency heads, and directory consistency all require some notion of observed state.

Auths-Proof responds in two ways:

1. assurance claims name what was actually established and at what time; and

2. local policy chooses whether historical or offline evidence is sufficient.

A future witness, transparency, or status adapter may strengthen assurance, but it remains evidence for principal control. It does not become an authority engine.

9. Related work

9.1 Proof-carrying systems

Proof-carrying code established the producer/consumer asymmetry: an untrusted producer supplies evidence, while the consumer runs a small checker against a local policy (Necula 1997). Proof-carrying authentication built a general higher-order logic in which requests arrive with proofs, and later web work demonstrated flexible distributed access control (Appel and Felten 1999; Bauer, Schneider, and Felten 2002).

Auths-Proof is less expressive and less formally ambitious. It does not ship arbitrary logical proofs. Its proof is a closed, signed grant-chain format with exact attenuation and action binding. The trade is intentional: limited expressiveness, bounded verification, straightforward cross-platform implementation, and a smaller semantic attack surface. A formal correspondence between its chain predicate and an authorization logic remains future work.

9.2 Authorization certificates and trust management

SPKI is the closest conceptual ancestor: authorization may bind directly to a key, delegation is explicit, validity is intersected, and heterogeneous certificate forms can reduce to a trusted intermediate representation (Ellison et al. 1999). PolicyMaker and decentralized trust management similarly separate credentials from application policy (Blaze, Feigenbaum, and Lacy 1996). The access control calculus formalizes principals speaking for themselves or others (Abadi et al. 1993).

Auths-Proof differs mainly in engineering boundary and artifact target. Its principal is opaque above an adapter; adapter output includes typed assurance; the action itself carries a body-bound proof; and the same verifier targets native and browser environments.

Macaroons support efficient decentralized attenuation through chained MACs and contextual caveats (Birgisson et al. 2014). They are compelling within a service secret domain. Auths-Proof instead uses asymmetric principal-control evidence and signed delegation across independently chosen identity methods, at the cost of larger proofs and public-key verification.

9.3 Centralized authorization services

Zanzibar demonstrates a globally consistent, highly available authorization service at enormous scale (Pang et al. 2019). Its relationship-based model and central checks solve a different deployment problem. Auths-Proof moves a bounded authorization chain with the action so a verifier can decide locally. The models can compose: a Zanzibar-like service could issue or validate a root grant, or an Auths application could require an online relationship check in addition to the portable proof. The paper does not claim that portable proofs replace globally consistent mutable policy.

9.4 Identity, transparency, and provenance

DID Core standardizes identifier syntax and document concepts while leaving method behavior to method specifications (Sporny et al. 2022). KERI uses key-event history, pre-rotation, and receipts to support self-certifying identifiers and key management (Smith 2019). CONIKS and Certificate Transparency show how authenticated data structures and public observation can make equivocation detectable (Melara et al. 2015; Laurie, Langley, and Kasper 2013).

Auths-Proof consumes such mechanisms as principal-control evidence. It does not define a universal DID method or transparency ledger. This separation is the point: better identity evidence should improve assurance without rewriting authority.

in-toto and Sigstore attach verifiable provenance to software supply-chain events (Torres-Arias et al. 2019; Newman, Meyers, and Torres-Arias 2022). They motivate application profiles for build, review, release, and deploy actions. Auths-Proof could bind who was allowed to initiate each action, while those systems preserve what happened to artifacts and who signed results.

9.5 Workload identity, OIDC, and message signatures

SPIFFE solves workload identity bootstrapping and rotation across heterogeneous infrastructure (Cloud Native Computing Foundation 2026). OIDC provides interoperable user authentication and claims (OpenID Foundation 2023). Both are plausible future principal-control or evidence adapters; neither should be caricatured as "only identity." Production systems build authorization around them. Auths-Proof's claim is narrower: authority can be made portable and independently verifiable without requiring every principal to use the same identity plane.

HTTP Message Signatures define canonical signing of selected HTTP components and explicitly leave application requirements to deployments (Backman, Richer, and Sporny 2024). An HTTP application profile could reuse that signing surface or carry an Auths proof beside it. Auths-Proof is transport-neutral and adds signed authority-chain semantics; HTTP Message Signatures are specific to HTTP message integrity and authenticity.

10. Discussion

10.1 What is actually new?

No individual ingredient is new: signed credentials, delegation chains, attenuation, local trust anchors, deterministic encodings, DIDs, transport abstraction, and portable verification all have prior art.

The potentially publishable systems idea is their disciplined composition:

- authority is one small protocol, not a feature of every identity adapter;
- principal-control mechanisms can differ at every hop in one chain;
- evidence quality survives verification as typed assurance;
- transport authentication and Auths authorization are a conjunction, never an implication;
- application semantics narrow the action surface without entering the kernel;
- native and browser verifiers share the same pure implementation.

The empirical claim must be demonstrated, not asserted. A stronger paper should add at least one independently implemented adapter, one independently implemented verifier, a formal model of the core invariants, and adversarial evaluation of parser and assurance boundaries.

10.2 The one primitive

The project began as a broader identity ecosystem. Its durable primitive is not "decentralized identity," "KERI tooling," "Git signing," "MCP security," or "key-addressed networking." Those are integrations.

> **Every action carries proof that it was authorized.**

This line is useful because it gives a stopping rule. If a feature does not help author, carry, verify, or safely apply that proof, it probably belongs in another repository or product.

10.3 Where to stop building

Auths-Proof should stop at:

```
proof format
+ authority semantics
+ principal–control verification port
+ bounded verifier
+ explicit context and assurance
+ conformance corpus
```

It should not absorb:

- a general network overlay;
- universal DID resolution;
- wallets, key backup, or HSM lifecycle;
- a secrets manager;
- an enterprise policy authoring suite;

- a global identity or revocation ledger;
- a generic MCP gateway;
- a deployment orchestrator;
- a relationship database;
- a universal audit warehouse.

Those are products or adapters built around the primitive. Keeping them outside is how the primitive remains credible.

10.4 Application opportunities

The architecture is most useful where actions cross trust or connectivity boundaries and must remain independently auditable:

- AI agent tool calls and delegated automation;
- software supply-chain approvals and release promotion;
- infrastructure change authorization;
- edge and intermittently connected device commands;
- cross-organization service actions;
- Git commit, tag, merge, and deploy authorization;
- data-access jobs and reproducible research pipelines;
- human approval delegated to short-lived machine actors.

The wedge is not "replace OIDC, SPIFFE, Vault, Iroh, or KERI." It is:

> Keep your principal and transport. Add portable, exact, independently verifiable authority to the action.

10.5 Research agenda

A credible academic program should test:

1. **Formal soundness.** Model grant attenuation, action binding, assurance, and transport non-interference in a proof assistant or executable specification.
2. **Independent interoperability.** Publish a language-neutral corpus and build a second verifier in a memory-safe language outside the Rust workspace.
3. **Adapter equivalence.** Specify when two different principal methods establish an equivalent control statement and when assurance prevents substitution.
4. **Freshness composition.** Evaluate witness, transparency, OCSP-like, and checkpoint evidence without putting network I/O in the kernel.
5. **Adversarial performance.** Measure worst-case bounded inputs, malformed encodings, long chains, multi-signature thresholds, WASM memory, concurrent service load, and WAN/relay transport.
6. **Usability.** Measure time to first authorized action, failure diagnosis, key-rotation recovery, and whether operators correctly distinguish `Denied` from `Indeterminate`.
7. **Profile safety.** Develop a review method that detects verify/execute mismatch and ambiguous resource naming before a profile ships.

11. Conclusion

Distributed systems have many strong ways to establish identity and secure a channel. They have fewer ways to make exact authority portable with an action without standardizing every participant on the same identity, algorithm, network, and application stack.

Auths-Proof explores a narrow answer: a deterministic proof-carrying authorization kernel with strict ports. KERI is one principal adapter. Raw keys, `did:key`, and `did:web` are others. Ed25519 and P-256 are V1 signature suites, not authority semantics. Iroh and memory are V1 transports, not the protocol. MCP is one application profile, not the product boundary.

The invariant that holds the composition together is simple:

Auths owns authority. Adapters prove principal control.

Every action carries proof that it was authorized.

The prototype shows that this separation is implementable across mixed principal methods, cryptographic suites, transports, and native/browser verifiers. The remaining work is substantial: formalization, independent implementations, hostile-input evaluation, freshness integration, and usable authoring. Those are also the tests that can turn an architectural thesis into a defensible systems result.

References

Abadi, Martín, Michael Burrows, Butler Lampson, and Gordon Plotkin. 1993. "A Calculus for Access Control in Distributed Systems." *ACM Transactions on Programming Languages and Systems* 15 (4): 706–34. https://doi.org/10.1145/155183.155225.

Appel, Andrew W., and Edward W. Felten. 1999. "Proof-Carrying Authentication." In *Proceedings of the 6th ACM Conference on Computer and Communications Security*, 52–62. https://doi.org/10.1145/319709.319718.

Backman, Annabelle, Justin Richer, and Manu Sporny. 2024. "HTTP Message Signatures." RFC 9421. RFC Editor. https://doi.org/10.17487/RFC9421.

Bauer, Lujo, Michael A. Schneider, and Edward W. Felten. 2002. "A General and Flexible Access-Control System for the Web." In *Proceedings of the 11th USENIX Security Symposium*, 93–108. USENIX Association. https://www.usenix.org/legacy/events/sec02/full_papers/bauer/bauer_html/.

Birgisson, Arnar, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. 2014. "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud." In *Network and Distributed System Security Symposium*. Internet Society. https://research.google/pubs/macaroons-cookies-with-contextual-caveats-for-decentralized-authorization-in-the-cloud/.

Blaze, Matt, Joan Feigenbaum, and Jack Lacy. 1996. "Decentralized Trust Management." In *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, 164–73. https://doi.org/10.1109/SECPRI.1996.502679.

Bormann, Carsten, and Paul Hoffman. 2020. "Concise Binary Object Representation (CBOR)." RFC 8949. RFC Editor. https://doi.org/10.17487/RFC8949.

Cloud Native Computing Foundation. 2026. "SPIFFE Overview." https://spiffe.io/docs/latest/spiffe-about/overview/.

Ellison, Carl M., Bill Frantz, Butler Lampson, Ron Rivest, Brian M. Thomas, and Tatu Ylonen. 1999. "SPKI Certificate Theory." RFC 2693. RFC Editor. https://doi.org/10.17487/RFC2693.

Haas, Andreas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. "Bringing the Web up to Speed with WebAssembly." In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 185–200. https://doi.org/10.1145/3062341.3062363.

Laurie, Ben, Adam Langley, and Emilia Kasper. 2013. "Certificate Transparency." RFC 6962. RFC Editor. https://doi.org/10.17487/RFC6962.

Melara, Marcela S., Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. 2015. "CONIKS: Bringing Key Transparency to End Users." In *24th USENIX Security Symposium*, 383–98. USENIX Association. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/melara.

n0 computer. 2026. "Iroh: Dial by Public Key." https://github.com/n0-computer/iroh.

Necula, George C. 1997. "Proof-Carrying Code." In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–19. https://doi.org/10.1145/263699.263712.

Newman, Zachary, John Speed Meyers, and Santiago Torres-Arias. 2022. "Sigstore: Software Signing for Everybody." In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2353–67. https://doi.org/10.1145/3548606.3560596.

OpenID Foundation. 2023. "OpenID Connect Core 1.0 Incorporating Errata Set 2." https://openid.net/specs/openid-connect-core-1_0.html.

Pang, Ruoming, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, et al. 2019. "Zanzibar: Google's Consistent, Global Authorization System." In *2019 USENIX Annual Technical Conference*, 33–46. USENIX Association. https://www.usenix.org/conference/atc19/presentation/pang.

Rundgren, Anders, Bret Jordan, and Samuel Erdtman. 2020. "JSON Canonicalization Scheme (JCS)." RFC 8785. RFC Editor. https://doi.org/10.17487/RFC8785.

Smith, Samuel M. 2019. "Key Event Receipt Infrastructure (KERI)." *arXiv Preprint arXiv:1907.02143*. https://doi.org/10.48550/arXiv.1907.02143.

Sporny, Manu, Amy Guy, Markus Sabadello, and Drummond Reed. 2022. "Decentralized Identifiers (DIDs) V1.0." W3C Recommendation. World Wide Web Consortium. https://www.w3.org/TR/did-core/.

Torres-Arias, Santiago, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. "In-Toto: Providing Farm-to-Table Guarantees for Bits and Bytes." In *28th USENIX Security Symposium*, 1393–1410. USENIX Association. https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.